PHP

What is PHP?
It is an open-source, server-side scripting language primarily used for web development.

echo and **print**?
echo can output multiple strings and does not return a value, whereas print can only output one string and always returns 1.

rules for naming variables?
Variables must start with a $ sign, followed by a letter or underscore. They are case-sensitive.

difference between == and ===?
== checks for value equality (with type coercion), while === checks for both value and identical data type.

include **and** require **differ?**
If a file is missing, include emits a warning and continues execution; require produces a fatal error and stops the script.

isset() and empty()?
isset() checks if a variable is declared and not null, while empty() checks if a variable is "empty" (e.g., "", 0, null, false, or an empty array).

Sessions vs. Cookies:
Sessions store data on the server, while cookies store it on the user's browser.

GET vs. POST Methods:
GET sends data via the URL (visible and limited size), whereas POST sends data in the request body (more secure and suited for large data).Get Default Limit is 2KB,2048 character

Traits?
A mechanism for code reuse in classes that reduces the limitations of single inheritance.

PDO vs. MySQLi:
PHP Data Objects (PDO) provides a consistent interface for multiple databases, whereas MySQLi is specific to MySQL databases.

Error Types:
PHP has four main error types:
1. **Notice** (non-critical),
2. **Warning** (serious but non-fatal),
3. **Fatal** (terminates script)
4. **Parse/Syntax Error** (prevents execution).

SOLID Principles:
These are five design principles used to make code more maintainable.
1. Single Responsibility
2. Open/Closed
3. Liskov Substitution
4. Interface Segregation
5. Dependency Inversion

Dependency Injection:
A design pattern where an object's dependencies are provided to it rather than the object creating them itself, improving testability.

What is OPcache?
A caching engine that stores precompiled script byte code in memory, significantly improving performance by removing the need to parse scripts on every request.

How to prevent SQL Injection?
Always use Prepared Statements and Parameterized Queries via PDO or MySQLi instead of concatenating user input directly into queries.

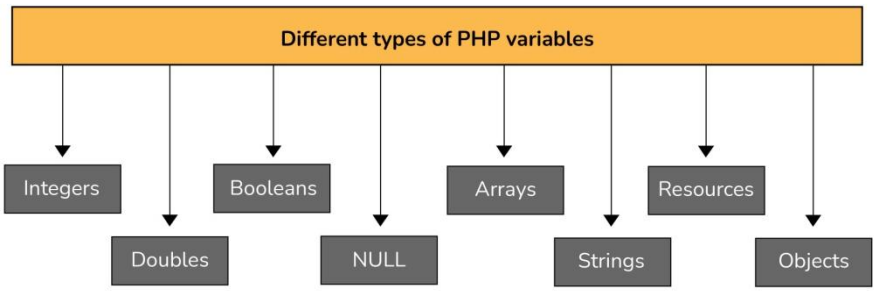
Magic Methods:
Special methods starting with __ that are automatically triggered by PHP during specific events.

1. __construct()
2. __call()
3. __get())

What does PEAR stands for?
PEAR stands for "PHP Extension and Application Repository". PEAR is a framework and repository for all of the reusable PHP components.
PEAR provides a higher level of programming for web developers. It contains all kinds of PHP code snippets and libraries. It also provides you with a command-line interface to automatically install packages.

Is PHP a case-sensitive language?
PHP can be considered as a partial case-sensitive language. The variable names are completely case-sensitive but function names are not. Also, user-defined functions are not case-sensitive but the rest of the language is case-sensitive.
For example, user-defined functions in PHP can be defined in lowercase but later referred to in uppercase, and it would still function normally.

What are the different types of variables present in PHP?



What is the meaning of a final method and a final class?
The final keyword in a declaration of the method indicates that the method cannot be overridden by subclasses. A class that is declared as final cannot be subclassed.
This is especially useful when we are creating an immutable class like the String class. Only classes and methods may be declared final, properties cannot be declared as final

Path Traversal
Path traversal is a form of **attack** to read into the files of a **web application**. '../' (dot-dot-sequences) is a cross-platform symbol to go up in the directory. Path traversal makes use of this symbol to operate the web application file. The attacker can reveal the content of the file attacked using the path traversal outside the root directory of a web server or application. It is usually done to gain access to secret passwords, tokens, and other sensitive information stored in the files.
Path Traversal is also called "Directory Traversal". It allows the attacker to exploit vulnerabilities present in the web file under attack.

What are cookies? How to create cookies in PHP?
Per site 20 cookies can be created in a single website or web app. 50 bytes is the initial size of the cookie and 4096 bytes is the maximum size of the cookie.

In PHP, we can create cookies using the setcookie() function:
setcookie(name, value, expire, path, domain, secure, httponly);

Parser in PHP?
A PHP parser is software that converts source code into the code that computer can understand. This means whatever set of instructions we give in the form of PHP code is converted into a machine-readable format by the parser.
You can parse PHP code with PHP using the token_get_all() function.

What is the purpose of @ in PHP?
In PHP, @ is used for suppressing error messages. If any runtime error occurs on the line which consists @ symbol at the beginning, then the error will be handled by PHP

disadvantages of PHP
The cons of PHP are:

1. PHP is not suitable for giant content-based web applications.
2. Since it is open-source, it is not secure. Because ASCII text files are easily available.
3. Change or modification in the core behavior of online applications is not allowed by PHP.
4. If we use more features of the PHP framework and tools, it will cause poor performance of online applications.
5. PHP features a poor quality of handling errors. PHP lacks debugging tools, which are needed to look for warnings and errors. It has only a few debugging tools in comparison to other programming languages.

Type hinting
(formally known as **type declarations**) in PHP allows you to explicitly specify the data type of function parameters, return values, and class properties. This feature transforms PHP from a purely loosely typed language into one that can enforce strict data contracts, making code more predictable and easier to debug.

Function Parameters: Enforces that arguments passed to a function match the declared type.
function setAge(int $age) { /* ... */ }

Return Values: (PHP 7.0+) Ensures a function returns the specific type promised.
function getStatus(): bool { return true; }

Class Properties: (PHP 7.4+) Restricts what values can be assigned to an object's properties.
class User { public string $name; }
OOPS

Classes and Objects:
A class acts as a blueprint, while an object is a specific instance.

Encapsulation & Access Modifiers:
Restricts access to data using public, protected, and private keywords.

Inheritance:
Enables child classes to inherit methods and properties from a parent using extends.

Abstraction & Interfaces:
Uses abstract classes and interfaces to define method structures that other classes must implement.
To use Interface use implement keyword

to use abstract class extend keyword use . atleast one method abstract in this class. The purpose of an abstract class is to enforce all derived classes (child classes) to implement the abstract method(s) declared in the parent class.

Constructors/Destructors:
__construct() initializes objects, while __destruct() handles cleanup.

Static Members & Traits:
Static properties/methods are accessed without instantiation; Traits allow code reuse across classes.

PHP - Access Modifiers
- public - the property or method can be accessed from everywhere. This is default
- protected - the property or method can be accessed within the class and by classes derived from that class
- private - the property or method can ONLY be accessed within the same class where they are defined

Class Constants
Class constants are useful if you need to define some constant data within a class.
A class constant has a fixed value, and cannot be changed once it is declared.
A class constant is declared inside a class with the const keyword.
The default visibility of class constants is public.
Class constants are case-sensitive. However, it is recommended to name the constants in all uppercase letters.

Abstract Classes and Methods
An abstract class is a class that contains at least one abstract method. An abstract method is a method that is declared, but not implemented in the abstract class. The implementation must be done in the child class (es).

Static Methods
The static keyword is used to create static methods and properties.

Static methods can be accessed directly - without creating an instance of the class first.

Namespaces
PHP namespaces are used to prevent naming conflicts between classes, interfaces, functions, and constants.

1. To avoid name conflicts, especially in larger projects
2. To organize code into logical groups
3. To separate your code from code in libraries
4. To allow the same name to be used for more than one class, without conflict

PHP Iterables
In PHP, an iterable is a value that can be looped through with a foreach() loop.
The iterable pseudo-type was introduced in PHP 7.1, and it can be used as a data type for function arguments and function return values.
An iterable can be an array or an object that implements the Iterator interface.
DB

Connecting PHP to MySQL
PHP supports two main extensions for working with MySQL databases:

- **MySQLi** (MySQL Improved)
- **PDO** (PHP Data Objects)

Example - MySQLi Object-OrientedGet your own PHP Server

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "mydb";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

Example - MySQLi Procedural

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "mydb";

// Create connection
$conn = mysqli_connect($servername, $username, $password, $dbname);

// Check connection
if (!$conn) {
  die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Example - PDO

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "mydb";

try {
  $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
  // set the PDO error mode to exception
```

```
  $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
  echo "Connected successfully";
} catch(PDOException $e) {
  echo "Connection failed: " . $e->getMessage();
}
?>
```

Close the Connection to MySQL
The connection will be closed automatically when the script ends. To close the connection before, use the following:
Example - MySQLi Object-Oriented:

```
$conn->close();
```

Example - MySQLi Procedural:

```
mysqli_close($conn);
```

Example - PDO:

```
$conn = null;
```

Get Last insertedID

```
$last_id = $conn->insert_id; //mysqli procedural
$last_id = mysqli_insert_id($conn); //mysqli oops
$last_id = $conn->lastInsertId(); // pdo
```

PHP MySQL Prepared Statements
PHP MySQL prepared statements are mainly used to prevent SQL injection attacks and to improve performance.
Prepared statements seperates the data from SQL commands.
Prepared statements basically work like this:

1. **Prepare:** An SQL query template with placeholders is sent to the server. The data values are not sent. Example: INSERT INTO MyGuests VALUES(?, ?, ?). Then, the server parses, compiles, and optimizes the SQL query template, without executing it
2. **Execute:** At a later time, the application binds the values to the parameters, and the database executes the query. The application may execute the query as many times as it wants with different values

Prepared statements have four main advantages:

1. **Reduced parsing time** - as the preparation on the query is done only once (although the statement is executed multiple times)
2. **Minimize bandwidth** - Bound parameters minimize bandwidth to the server as you need send only the parameters each time, and not the whole query
3. **Security** - Prepared statements are very useful against SQL injections, because parameter values, which are transmitted later using a different protocol, need not be correctly escaped. If the original statement template is not derived from external input, SQL injection cannot occur
4. **Cleaner code** - by seperating data from SQL commands

The PHP SimpleXML Parser
The SimpleXML Parser allows us to easily read and manipulate XML data. It is a lightweight parser and use as few resources as possible. The SimpleXML Parser is ideal for small-sized, uncomplicated XML files.
This parser provides an easy way to access elements, attributes and textual content of an XML document.
You can read XML from a string or from a file, using these functions:

- simplexml_load_string() - Reads XML data from a string
- simplexml_load_file() - Reads XML data from a file

PHP

The PHP DOM Parser
The DOM parser makes it possible to parse XML documents in PHP.
The DOM parser is a tree-based parser and loads the entire document in memory as a tree structure. It analyzes the whole document, and provides access to the tree elements (DOM).
A tree-based parser is ideal for smaller XML documents, but not for large XML documents, as it can be very memory intensive!
Look at the following XML document fraction:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Tove</to>
```

The DOM parser sees the XML above as a tree structure:

- Root element: <note>
- Child element: <to>
- Text element: "Tove"

DOM Parser - Load and Output XML

The XML file "note.xml" will be used in the examples below.
The PHP DOM parser uses the DOMDocument class to parse documents into a Document Object Model (DOM) structure.
In the following example we initialize the DOM parser, and loads the XML from "note.xml" into it. Then the saveXML() function saves the XML document into a string, so we can output it:

```
<?php
// Initialize DOM parser
$xmlDoc = new DOMDocument();

// Load XML file
$xmlDoc->load("note.xml");

// Saving all the document
print $xmlDoc->saveXML();
?>
```

Array create an indexed array named $cars, assign three elements to it, and then print a text containing the array values:

```
$cars=array("Volvo","BMW","Toyota");
```

The **array_change_key_case()** function changes all keys in an array to lowercase or uppercase.

```
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43");
print_r(array_change_key_case($age,CASE_UPPER));
```

The **array_chunk()** function splits an array into chunks of new arrays.

```
$cars=array("Volvo","BMW","Toyota","Honda","Mercedes","Opel");
print_r(array_chunk($cars,2));
```

array_column()

```
// An array that represents a possible record set returned from a database
$a = array(
  array(
    'id' => 5698,
    'first_name' => 'Peter',
    'last_name' => 'Griffin',
  ),
  array(
    'id' => 4767,
    'first_name' => 'Ben',
    'last_name' => 'Smith',
  ),
  array(
    'id' => 3809,
    'first_name' => 'Joe',
    'last_name' => 'Doe',
  )
);

$last_names = array_column($a, 'last_name');
print_r($last_names);
```

Output:

```
Array
(
  [0] => Griffin
  [1] => Smith
  [2] => Doe
)
```

array_combine() Create an array by using the elements from one "keys" array and one "values" array:

```
$fname=array("Peter","Ben","Joe");
$age=array("35","37","43");
$c=array_combine($fname,$age);
print_r($c);
```

array_flip() Flip all keys with their associated values in an array:

```
$a1=array("a"=>"red","b"=>"green","c"=>"blue","d"=>"yellow");
$result=array_flip($a1);
print_r($result);
```

Output

Array ([red] => a [green] => b [blue] => c [yellow] => d)

The **array_keys()** function returns an array containing the keys.

```
$a=array("Volvo"=>"XC90","BMW"=>"X5","Toyota"=>"Highlander");
print_r(array_keys($a));
```

The **array_merge()** function merges one or more arrays into one array.

```
$a1=array("red","green");
$a2=array("blue","yellow");
print_r(array_merge($a1,$a2));
```

The array_pop() function deletes the last element of an array.

```
$a=array("red","green","blue");
array_pop($a);
print_r($a);
```

The array_push() function inserts one or more elements to the end of an array.

```
$a=array("red","green");
array_push($a,"blue","yellow");
print_r($a);
```

The in_array() function searches an array for a specific value.

```
$people = array("Peter", "Joe", "Glenn", "Cleveland");

if (in_array("Glenn", $people))
  {
  echo "Match found";
  }
else
  {
  echo "Match not found";
  }
```

The **count() function** returns the number of elements in an array or in a countable object.

```
$cars=array("Volvo","BMW","Toyota");
echo count($cars);
```

The each() function returns the current element key and value, and moves the internal pointer forward.

```
$people = array("Peter", "Joe", "Glenn", "Cleveland");
print_r (each($people));
```

- current() - returns the value of the current element in an array
- end() - moves the internal pointer to, and outputs, the last element in the array
- next() - moves the internal pointer to, and outputs, the next element in the array
- prev() - moves the internal pointer to, and outputs, the previous element in the array
- reset() - moves the internal pointer to the first element of the array

PHP String Functions

The **strlen()** function returns the length of a string.
echo strlen("Hello world!");

The str_word_count() function counts the number of words in a string.
echo str_word_count("Hello world!");

The **str_contains()** function checks if a string contains a specific substring. Function is only available in PHP 8.0 and later. For older versions, use the strpos() function. If a match is found, the function returns a boolean true. If no match is found, it will return a boolean false.
$txt = "I really love PHP!";
var_dump(str_contains($txt, "love"));
Note: This function performs a case-sensitive search.

The **strpos()** function searches for a specific text within a string.
If a match is found, the function returns the character position of the first match. If no match is found, it will return false.
Note: This function performs a case-sensitive search.
echo strpos("Hello world!", "world");
Tip: The first character position in a string is 0 (not 1).

The **str_starts_with()** function checks if a string starts with a specific substring. The str_starts_with() function is only available in PHP 8.0 and later. If a match is found, the function returns a boolean true. If no match is found, it will return a boolean false.
$txt = "I really love PHP!";
var_dump(str_starts_with($txt, "I really"));
Note: This function performs a case-sensitive search.
The following example will return a boolean false, because "i really" is not found in the main string:

The **str_ends_with()** function checks if a string ends with a specific substring. The str_ends_with() function is only available in PHP 8.0 and later. If a match is found, the function returns a boolean true. If no match is found, it will return a boolean false.
$txt = "I really love PHP!";
var_dump(str_ends_with($txt, "PHP!"));
Note: This function performs a case-sensitive search.
The following example will return a boolean false, because "php!" is not found in the main string:

The **explode()** function splits a string into an array.
The first parameter of the explode() function represents the "separator". The "separator" specifies where to split the string.
$x = "Hello lovely World!";
$y = explode(" ", $x);
print_r($y);

The **substr()** **function** is used to extract a part of a string (slice a string).
Specify the start index and the number of characters you want to return.
$x = "Hello World!";
echo substr($x, 6, 5);

As of April 2026, the latest stable version of PHP is **8.5** (specifically version 8.5.5, released April 9, 2026).
Released on November 20, 2025, PHP 8.5 introduces new features like the Pipe operator (|>), array_first()/array_last() functions, and native opcache, offering significant performance improvements over older versions

Introduced in **PHP 8.2**, a **readonly class** is a shorthand for declaring every property within that class as readonly. This feature is primarily used for creating immutable data structures like Data Transfer Objects (DTOs) or Value Objects.

Key Characteristics
Automatic Immutability: Marking a class as readonly automatically applies the readonly modifier to all of its instance properties.
Initialization Rule: Properties can only be initialized **once**, typically within the constructor. Once set, they cannot be modified or unset.
Mandatory Typing: Every property in a readonly class **must be typed** (e.g., string, int, or mixed).
No Dynamic Properties: Dynamic properties (properties created at runtime) are strictly forbidden and will throw an Error.
No Static Properties: readonly classes cannot contain static properties as they cannot be made readonly.

Inheritance Rules
Strict Matching: A readonly class can only extend another readonly class.
Exception (PHP 8.3+): Starting with **PHP 8.3**, the restriction was slightly loosened to allow readonly classes to extend certain non-readonly classes, provided they follow strict immutability semantics.

PHP 8.5 feature :
Pipe Operator (|>): Enables clean, readable chaining of functions, passing the return value of the left expression to the right.

```
$stug = "PHP 8.5 release!"
$slug = $stug |> trim(…)
|> (fn($str)=>str_replace(` `,'-`,$str))
//Output
var_dump($stug);
```

Clone With: Allows cloning an object while simultaneously modifying specific properties in a concise syntax. This enables straight forward support of the "with-er" pattern for readonly classes.

Array Functions: New **array_first() and array_last()** functions provide easy retrieval of the **first and last** elements, working for both indexed and associative arrays.

Syntax Improvements:
- **No Discard Attribute:** Allows flagging functions where the result should not be ignored to prevent bugs. **#[!NoDiscard]**

- **Clone Logic Alteration:** Allows modifying properties during object cloning.
- **Static Property Visibility:** Expands asymmetric visibility to static properties.

The **OpCache** extension is now a non-optional part of PHP, and will always be present in the binary, regardless of how packagers package and distribute PHP.

#[Deprecated] can now be used with traits (not just classes and interfaces), as well as constants (since constants cannow have attributes).

#[Override] can now target class properties; this will cause the engine to validate that a property of the same name exists in a parent class or implemented interface, and emit a compilation error if it does not.

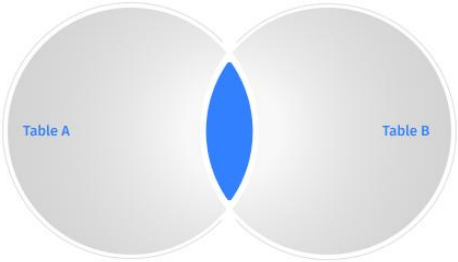
You can now enable backtraces for fatal errors. When enabled, if you catch errors, will include a trace key with the associated backtrace. This feature will make it easier to identify the root cause of a fatal error.

setcookie() and setrawcookie() will now accept "secure" and "partitioned" options, allowing you to control Cookies Having Independent Partitioned State (CHIPS) behavior.
Additionally, session_start() and session_set_cookie_params() allow you to enable cookie partitioning for the session cookie.

Attributes can now be applied to constants.

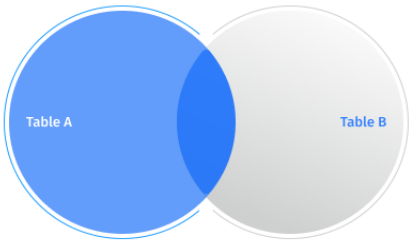
DB
INNER JOIN returns rows with matching values across all specified tables.

INNER JOIN



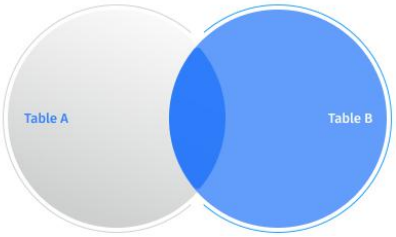
LEFT or [OUTER] JOIN includes all rows from the left table and only matching rows from the right table. Non-matching rows from the right table are filled with NULLs.

LEFT OUTER JOIN



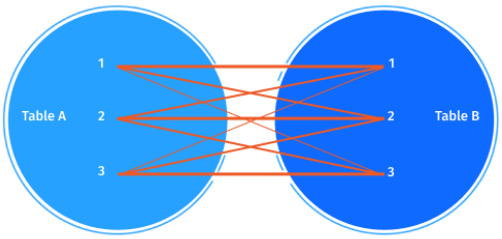
RIGHT or [OUTER] JOIN returns all rows from the right table and only the rows from the left table that meet the JOIN condition. Non-matching rows from the left table are filled with NULLs.

RIGHT OUTER JOIN



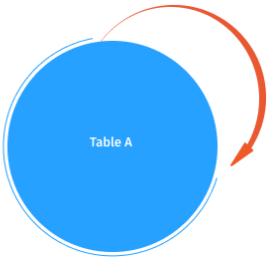
CROSS JOIN combines every row from one table with each row from another, resulting in a table with all possible row combinations.

CROSS JOIN



SELF JOIN allows comparisons within the same table or the extraction of hierarchical data. Table aliases are used here to avoid repeating the same table name.

SELF JOIN



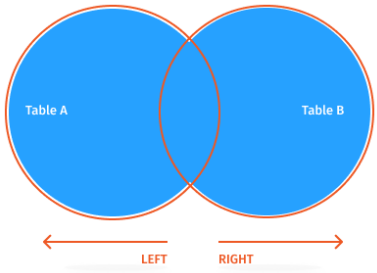
FULL OUTER JOIN in SQL Server the combination of LEFT OUTER JOIN and RIGHT OUTER JOIN with the UNION operator can be used in MySQL to get the results similar to FULL OUTER JOIN in SQL Server.
This combination returns all rows from both tables involved in the JOIN query, matching rows from one table with corresponding rows in the other table where possible.

Though MySQL does not support FULL OUTER JOIN (as opposed to SQL Server, for instance), it offers alternatives: LEFT OUTER JOIN and RIGHT OUTER JOIN.

```
SELECT * FROM tableA
LEFT JOIN tableB ON tableA.id = tableB.id
UNION
SELECT * FROM tableA
```

RIGHT JOIN tableB ON tableA.id = tableB.id

FULL JOIN



Core Normal Forms
Normalization is progressive; each stage (Normal Form) builds upon the rules of the previous one.

First Normal Form (1NF) - Atomicity:
Ensures each column contains only atomic (indivisible) values.
Rule: No multi-valued attributes (e.g., a comma-separated list of phone numbers in one cell) or repeating groups.

Second Normal Form (2NF) - No Partial Dependency:
Must satisfy 1NF rules.
Rule: All non-key attributes must depend on the **entire** primary key, not just a part of it. This is only applicable when using composite keys.

Third Normal Form (3NF) - No Transitive Dependency:
Must satisfy 2NF rules.
Rule: Non-key attributes should not depend on other non-key attributes. For example, if a ZipCode determines the City, they should be moved to a separate table to avoid duplicating the city for every address in that zip code.

The **FIND_IN_SET()** **function** in MySQL is used to find the position of a string within a comma-separated list of strings. It is particularly useful for searching within columns that store values as comma-delimited text or when working with the MySQL SET data type.

FIND_IN_SET(string, string_list)
string: The value you want to search for.
string_list: A string containing values separated by commas (e.g., 'a,b,c,d').

Any :
In MySQL, **ANY** is a logical operator used in WHERE or HAVING clauses to compare a single value against a set of values returned by a subquery.
1. **Evaluation:** It returns TRUE if the comparison is true for **at least one** value in the subquery.
2. **Comparison Operators:** It must be preceded by a standard comparison operator such as =, >, <, >=, <=, or <> (not equal).
3. **Synonym:** In MySQL, the keyword **SOME** is an

Example:
SELECT ProductName FROM Products
WHERE ProductID = ANY (SELECT ProductID FROM OrderDetails WHERE Quantity > 10);

Core Storage Engines
InnoDB: The default storage engine since MySQL 5.5. It is **transaction-safe (ACID compliant)** and supports **row-level locking**, foreign keys, and automatic crash recovery. It is recommended for most general-purpose applications.
MyISAM: The former default engine. It is **non-transactional** and uses **table-level locking**, making it faster for read-heavy workloads but less reliable for concurrent writes.
MEMORY: Stores all data in **RAM** for extremely fast access. However, data is **volatile** and lost upon server restart.
CSV: Stores data in plain-text comma-separated values (CSV) format. It is primarily used for easy data exchange with external applications like spreadsheets.
ARCHIVE: Optimized for storing large amounts of **rarely accessed historical data**. It uses compression to save disk space and supports only INSERT and SELECT operations.

PYTHON

What is Python?
It is powerful, general purpose, high level, object oriented programming language. It's developed by Guido Van Rossum in 1991.

Benefits of Python ?
1. It is very simple and easy to learn.
2. It powerful, fast and secure.
3. It has very simple syntax.
4. It is powerful scripting language.
5. It can be run on different kind of platform like Windows , Mac, Linux, etc.,

Dynamically Typed Language?
Python is a **dynamically typed language**, meaning the interpreter performs type checking only during execution (at runtime). Unlike statically typed languages like Java or C++, variables in Python are not bound to a specific data type at declaration

What are the applications developed using Python?
1. Web Application
2. Software Development
3. Database GUI application
4. Scientific and Numeric Computing
5. Business Applications
6. Console Based Application

What are the features in Python?
1. Independent Platform
2. Object oriented
3. Flexible
4. Structure oriented
5. Portable
6. Simple

Is python, case sensitive language or not?
Yes, python is a case sensitive language

Why python is called an interpreted language?
An interpreted language is any programming language which is not in machine level code before run time. Therefore, python is an interpreted language.
Execute statements line by line . No intermediary compilation step.

Why indentation is required in Python?
Indentation is necessary for Python. It specifies a block of code. All code within loops, classes, functions, etc. And also it is specified within the indented block. It is usually done by using four space characters. If your code is not indented necessarily, it will not execute accurately and will throw errors as well.

What is an iterator in Python?
An iterator is an object which implements the iterator protocol.
It has a __next__() method which returns the next item in iteration, also iterators are objects which can iterate objects like list, string, etc.

PEP8 and Why it is important?
PEP 8 is the official Style Guide for Python Code, providing a set of conventions to improve readability and consistency across the Python ecosystem. It is important since it documents the style guidelines for python code.

What is comment?
Python comments are statements that are not executed by the compiler.
It is not considered as a part of program.

What are the types of comments in Python?

There are two types of comment in Python.

1. Single line comment
2. Multi line comment

What is single line comment?

1. It is used to comment only one line.
2. Hash (#) is used for single line comment.

What is multiline comment?

1. It is used to comment a block of code.
2. Triple quotes are used to multiline comment starts with ‘’’ and ends with ‘’’.

Scope in python

Scope determines the visibility and lifetime of a variable. It defines which parts of your program can "see" or access a specific name (variable, function, etc.). Python follows the LEGB Rule, which is the order in which the interpreter looks up names.

The LEGB Rule if you reference a variable, Python searches these four scopes in this exact order:

1. Local (L) Variables : defined inside a function. They exist only while the function is running.
2. Enclosing (E) Variables : in the local scope of outer functions (relevant in nested functions/closures).
3. Module level/ Global (G) Variables : defined at the top level of a script or module, or declared global.
4. Outermost/ Built-in (B) : Names pre-loaded into Python, like print(), len(), or ValueError.

List []	**Tuple ()**
Mutable (can add, remove, or change)	**Immutable** (fixed after creation)
Consumes more memory	Consumes less memory
Slightly slower	Slightly faster
Many (append, pop, sort, etc.)	Few (count, index only)
Collection of items that change	Constant data, "Records"
No (cannot be a dictionary key)	Yes (can be a dictionary key)

Common built in data type in Python?

Several built in data type in python.

Python does not require data type to be defined explicitly during variable declarations type errors are likely to occur if the knowledge of data type and their comp ability with each other are neglected.

--type() && instance() function to check the type of the variable

Here are the most common built-in data types grouped by category:

1. Numeric Types

Used to represent numbers.

Type	**Name**	**Example**	**Description**
int	Integer	10, -5, 0	Whole numbers of unlimited length.
float	Floating-point	3.14, -0.001	Numbers with a decimal point (64-bit precision).
complex	Complex	2 + 3j	Numbers with a real and imaginary part.

2. Sequence Types

Used to store collections of items in a specific order.

Type	Name	Example	Mutability
str	String	"Hello"	**Immutable** (Text data).
list	List	[1, "a", 3.5]	**Mutable** (Changeable collection).
tuple	Tuple	(1, 2, 3)	**Immutable** (Fixed collection).
range	Range	range(10)	**Immutable** (Sequence of numbers).

3. Mapping Type

Used to store data in **key-value** pairs.

Type	Name	Example	Description
dict	Dictionary	{"id": 1, "name": "Gemini"}	Fast lookups using unique keys.

4. Set Types

Used for collections of **unique** items.

Type	Name	Example	Mutability
set	Set	{1, 2, 3}	**Mutable** (No duplicates).
frozenset	Frozen Set	frozenset({1, 2})	**Immutable** version of a set.

5. Boolean & None

Used for logic and representing "nothing."

Type	Name	Example	Description
bool	Boolean	True, False	Result of logical comparisons.
NoneType	None	None	Represents the absence of a value.

Modules?

In Python, a **module** is simply a file containing Python code (functions, classes, and variables) that ends in the .py extension.

Think of a module like a **toolbox**. Instead of keeping every single tool you own on your workbench (which would make it cluttered and impossible to manage), you keep specific tools in separate boxes and only pull out the one you need when you need it.

Advantage:

- **Maintainability:** You can break large programs into smaller, organized files.
- **Reusability:** Write a function once in a module and use it across many different projects.
- **Name spacing:** Modules help avoid "name clashes." You can have a function named calculate() in two different modules without them interfering with each other.

2. How to Use Modules

You bring a module into your current script using the import statement.

Callable?

In Python, a **callable** is any object that can be called using parentheses () and potentially some arguments.

Think of it as anything that behaves like a function. If you can put () after it without getting a TypeError, it's a callable.

What counts as a Callable?

- Built-in Functions and Methods
- User-Defined Functions
- Classes
- Objects with a __call__ method

How to check if something is Callable?

Python provides a built-in function called callable() that returns True or False.

Keyword?

The word which is predefined in the library is called keyword.

- Keywords cannot use as a variable function name, class name or any other identifier.

What is Float function?

- Float () is a predefined function which is used to convert given data into float.

What is int() function?

- int() is a predefined function which is used to convert string into integer.

What is print in Python?

- It is a predefined function which is used to print data or information.

How can you convert a number to a string?

- In order to convert a number into a string, use the inbuilt function str(). If you want a octal or hexadecimal representation, use the inbuilt function oct() or hex().

Pass in python?

In Python, the pass statement is a **null operation**. When it executes, nothing happens.
It is used as a in situations **placeholder** where the Python syntax requires a statement to be present, but you aren't ready to write the logic yet.

What is an Operator?

- It is a special symbol which is used to perform logical or mathematical operation on data or variable

What are the types of operator in Python?

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Bit-wise operators
- Membership operators
- Identity operators

What is Operand?

- It is a data or variable on which the operation is to be performed

What is the use of // operator in Python?

- It is a Floor Division operator, which is used for dividing two operands with the result as a quotient showing only digits before the decimal point. For instance, 10//5 = 2 and 10.0//5.0 = 2.0

What is the purpose of relational operators in Python?

- The purpose of relational operators in Python is to compare values.

What are assignment operators in Python?

- The assignment operators in Python can help in combining all the arithmetic operators with the assignment symbol.

What are membership operators?

- With the operators 'in' and 'not in'
  print('me' in 'disappointment')

  o/p: True

How to differentiate Identity operators & Membership operators?

- Unlike membership operators, the identity operators compare the values to find out if they have the same value or not
- While the terms are often used interchangeably, there is a distinct structural difference between a **module** and a **package**.

- The simplest way to think about it: **A module is a file; a package is a folder.**

Core Difference Module/Packages

Module	Package
A single .py file.	A directory (folder) containing modules.
Functions, classes, and variables.	Multiple modules and sub-packages.
To group related code in one file.	To group related modules together.
Just a .py extension.	Historically required an __init__.py file.

Global, private & protected attribute in Python?
In Python, the concept of **public**, **protected**, and **private** attributes is handled differently than in languages like Java or C++. Python doesn't have "enforced" access modifiers; instead, it relies on **naming conventions** and a bit of "name mangling."

Global :
Global variables aren't attributes of a class; they are variables defined at the top level of a module. They are accessible to any function or class within that module.

- **To Read:** Just use the name.
- **To Modify inside a function:** Use the global keyword.

Protected Attributes
A protected attribute is intended to be used only within the class and its subclasses. However, Python does **not** actually prevent outside access. The single underscore is a signal to other developers: *"Hey, this is internal; touch it at your own risk."*

- **Naming:** Single underscore prefix (e.g., self._balance).
- **Access:** Technically accessible everywhere, but should be avoided outside the class hierarchy.

Private Attributes
Private attributes are intended to be hidden from everyone except the class itself. Python "protects" these by changing their name internally so they aren't easily found. This is called **Name Mangling**.

- **Naming:** Double underscore prefix (e.g., self.__password).
- **Access:** Cannot be accessed directly by the name __password from outside.

Public Attributes
By default, every attribute or method you create in a class is **public**. It can be accessed and modified from anywhere—inside the class, outside the class, or in a subclass.

- **Naming:** No underscores (e.g., self.name).
- **Access:** Direct.

Self in Python ?
In Python, self represents the **instance of the class**. By using the self keyword, you can access the attributes and methods of the class in Python. It binds the attributes with the given arguments.
The reason you need self is that Python does not use the @ syntax (like Ruby) or special internal scoping to refer to instance attributes.

What is __INIT__ in python?
In Python, __init__ is a special method known as a **constructor**. It is automatically called when you create a new instance (object) of a class.
Its primary job is to **initialize** the object's attributes, setting up its initial state.
How it Works:
The __init__ method is part of what Python calls "Magic Methods" (or Dunder methods, short for **D**ouble **Under**score). You don't call it directly like obj.__init__(); instead, it triggers the moment you use the class name to create an object.

What is import method?

- Import is a keyword, to access the script from another python file or module.

What is Input?

- Input () is a predefined function which is used to take user input in python. • Default user input is of type string

What is "if statement"?

- If the condition is true its body will execute otherwise does not execute

What is meant by for loop?

- For loop is used for sequential traversal.it can be used to traverse string or array. For loop is used to iterate over a sequence list, string, tuple, etc., Iterating over a sequence is called traversal.

What is clear () function in list?

- This function is used to empty the list.

How can you count duplicate elements in a given list?

```
Count duplicate elements in a given list
list1 = [2, 3, 4, 3, 10, 3, 5, 6, 3]
e =  list1.count(3)
print('The count of element: 3 is ', e)
```

What is meant by continue statement?

- It is used to skip the next statement and continue the loop. This mostly used with loop.

Break?

Terminate the loop

Unit test in python

In Python, **Unit Testing** is a software testing method where individual units of source code (functions, methods, or classes) are tested to determine if they are fit for use. The goal is to isolate each part of the program and show that the individual parts are correct.
Python comes with a built-in library called unittest to help you write and run these tests. Alternative: pytest
While unittest is built-in and powerful, many Python developers prefer an external library called **pytest**. It is much more "Pythonic" and requires less boilerplate code.

Docstring

In Python, a **docstring** (documentation string) is a literal string used to document a specific segment of code, such as a function, class, or module.
Unlike regular comments (#), docstrings are stored as an attribute of the object they describe, allowing them to be inspected at runtime.

What is difference between python Arrays and Lists?

Arrays:

- Arrays in python can only contain same data type of elements.
- Homogeneous
- Consumes far less memory that lists

Lists:

- List in python can contain elements of different data types.
- Heterogeneous
- Consuming large memory

Memory Manage in Python?

In Python, memory management is an automated process handled by the **Python Memory Manager**. You don't have to manually allocate or deallocate memory (like you would in C or C++), which prevents many common bugs like memory leaks.
Python uses two primary mechanisms to manage memory: **Reference Counting** and **Garbage Collection**.

Python divides memory into two main areas:

- **Stack Memory:** Stores "static" data—function calls, local variables, and references to objects. It follows the Last-In-First-Out (LIFO) order.

- **Private Heap:** This is where the actual **objects** (lists, strings, dictionaries) live. The Python Memory Manager controls this heap through an API.

NameSpace:
A **namespace** in Python is essentially a "naming system" that ensures all the names in your program (variables, functions, classes, etc.) are unique and don't conflict with each other.

Think of a namespace as a **dictionary** where the **keys** are the names you've defined and the **values** are the objects they point to.

Types of Namespaces

Namespace	Lifetime	Description
Local	During function execution.	Names created inside a function. Deleted when the function returns.
Global	Until the script ends.	Names defined at the top level of a module/file.
Built-in	For the entire program life.	Names like print, id, and len that are always available.

Scope vs. Namespace
These two are often confused:
- **Namespace:** The physical "container" (dictionary) where the names live.
- **Scope:** The **rule** that defines which namespace you are allowed to look into and in what order (see the **LEGB rule** we discussed earlier).

Scope Resolution in Python?
Sometime the object within the same scope have the same name but different functionality .
In such cases scope resolution comes into picture which resolve that which function call.
Math & Cmath have a lot of function that are common to both of them.

Eg : - log10() , acos(), exp()

It is necessary to resolve this ambiguity add prefix with them add their module name like
Math.log10() & cmath.log10()
Decorators:

In Python, a **decorator** is a design pattern that allows you to modify or extend the behavior of a function or class without permanently changing its source code.

Think of a decorator like **gift wrap**. You have a gift (the function), and you wrap it in paper (the decorator) to change how it looks or adds a little extra flair before the person opens it.

Decorators are applied using the @ symbol followed by the decorator name, placed right above a function definition.

```
def my_decorator(func):
    def wrapper():
    print("--- Something is happening before the function is called. ---")
     func()
    print("--- Something is happening after the function is called. ---")
return wrapper

@my_decorator
def greet():
    print("Hello, world!")

greet()
```

Lambda?
A lambda is an anonymous function. This function can have any number of parameters but, can have just one statement.

Example: X=lambda a,b:a8b

```
Print(X(2,4))
```

It is generally used in situation requiring an anonymous function for short time period.

Lambda functions can be used in either of the two ways:

1) Assigning lambda function to variable:
e.g:
```
Mul = lambda a,b : a*b
Print (Mul)
```

2) Wrapping lambda fuction inside other function :
e.g:
```
def mywrapper(n):
    return lambda a,b :a*b

Mul = mywrapper()
Print(Mul(2));
```

Copy :
In Python, there is a big difference between **assigning** a variable and **copying** one. If you simply use the = sign, you aren't creating a new object—you're just creating a new name that points to the same object.
To actually create a duplicate, Python provides the copy module with two options: **Shallow Copy** and **Deep Copy**.

Shallow Copy

A shallow copy creates a new collection object, but it **does not** create copies of the objects inside it. It just copies the *references* to those objects.

- **How to do it:** copy.copy(obj) or slicing list[:].
- **The catch:** If the list contains other lists (nested), the inner lists are still shared.

```
import copy
old_list = [[1, 2], [3, 4]]
new_list = copy.copy(old_list)
new_list[0][0] = 'X'
print(old_list)
# Outputs: [['X', 2], [3, 4]] — The change leaked!
```

Deep Copy

A deep copy creates a new collection object and then **recursively** creates copies of everything found inside it. The two objects are now completely independent.

```
import copy
old_list = [[1, 2], [3, 4]]
new_list = copy.deepcopy(old_list)
new_list[0][0] = 'X'
print(old_list) # Outputs: [[1, 2], [3, 4]] — Perfectly safe.
```

Xrange vs Range?
In short: **xrange** was a Python 2 feature designed for memory efficiency. In Python 3, xrange was removed and renamed to **range**.

Comparison Summary

Feature	Python 2 range	Python 2 xrange	Python 3 range
Type	List	xrange object	range object
Memory	High (stores all items)	Low (calculates on fly)	Low (calculates on fly)
Speed (Large N)	Slow (init)	Fast (init)	Fast (init)
Slicing	Returns a list	Returns an xrange	Returns a range

XRange has been deprecated as of python3.x. Now range does exactly the same as what xrange used to do in

python 2.x.

Pickling VS unpickling?
Pickling and **Unpickling** are the processes used to convert Python objects into a format that can be stored or transmitted, and then reconstructed later. This is often called **serialization** and **deserialization**.

Pickling (Serialization)

Pickling is the process of converting a Python object (list, dict, class instance, etc.) into a **byte stream**. This byte stream can be saved to a file or sent over a network.

- **Module:** pickle
- **Method:** pickle.dump() (to a file) or pickle.dumps() (to a string).

```
import pickle
data = {'name': 'Gemini', 'version': 3.0, 'active': True}
# Pickling: Writing to a binary file with open('data.pkl', 'wb') as file:
pickle.dump(data, file)
```

Unpickling (Deserialization)

Unpickling is the inverse operation. It takes a byte stream and converts it back into a functional Python object in your memory.

- **Method:** pickle.load() (from a file) or pickle.loads() (from a string).

```
# Unpickling: Reading from the binary file
with open('data.pkl', 'rb') as file:
    reconstructed_data = pickle.load(file)

print(reconstructed_data['name'])  # Outputs: Gemini
```